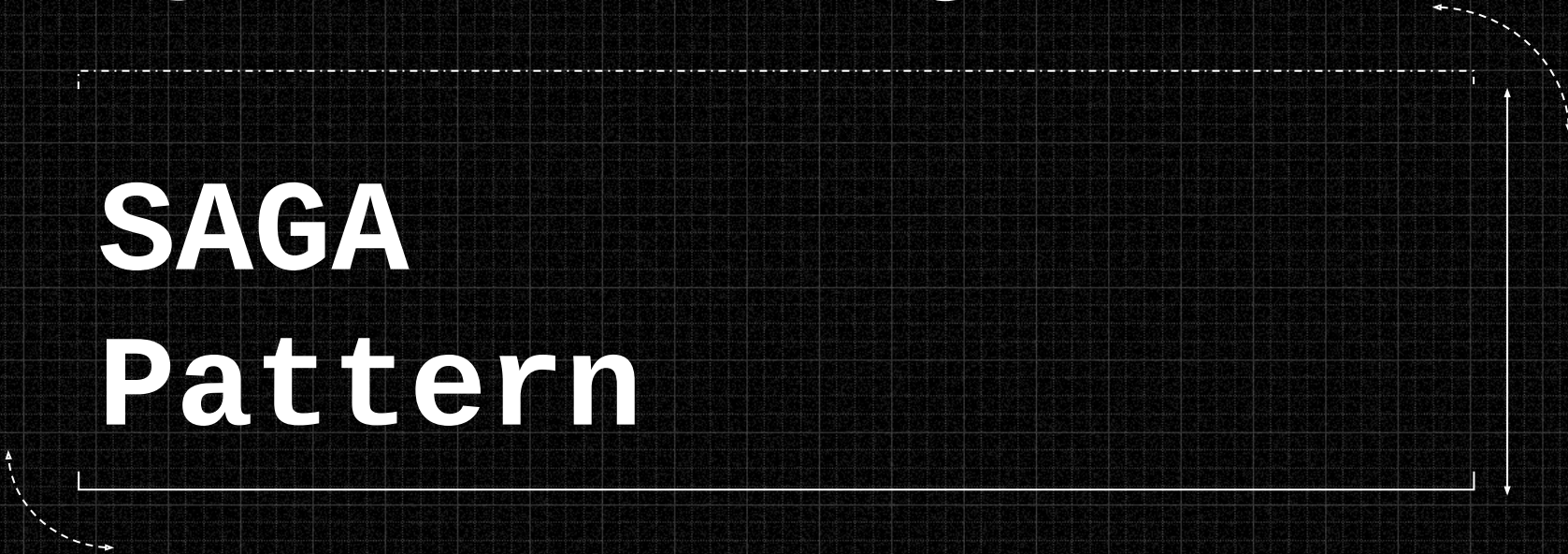


System Design

SAGA Pattern



\$ whoami

Matheus Fidelis

Engenheiro de \$RANDOM

@fidelissauro

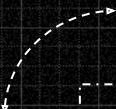
<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>



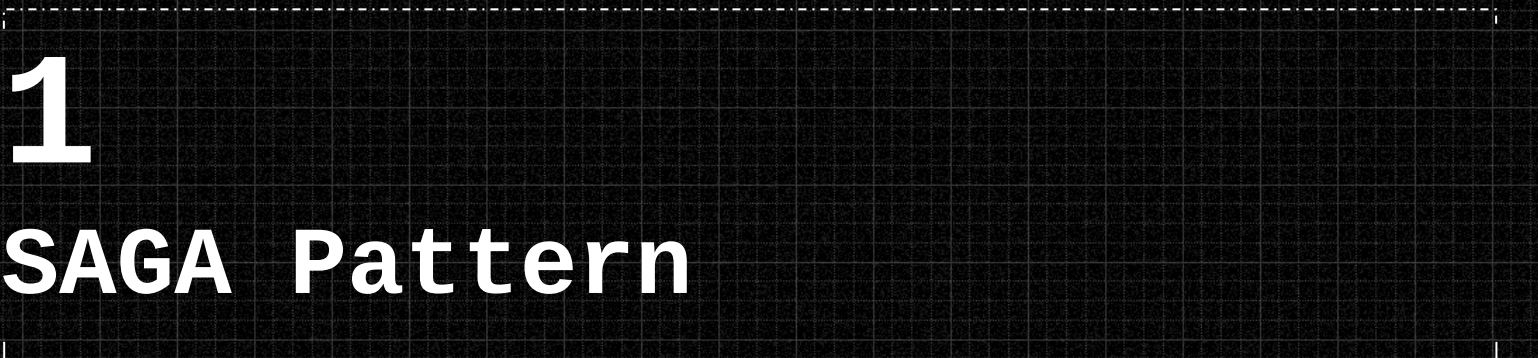
OBJETIVOS

- ✓ Definições
- ✓ Consistencia em Sistemas Distribuidos
- ✓ Transações de Longa Duração
- ✓ Tipos de SAGA
- ✓ Tradeoffs



1

SAGA Pattern



Definições e
Conceitos



Contexto e Motivação do Saga Pattern

- Consistencia em Múltiplos Serviço
- Sistemas distribuídos introduzem falhas parciais como estado normal
- Transações deixam de ser locais e passam a cruzar múltiplos domínios
- ACID tradicional não escala para microserviços
- Necessidade de consistência sem bloqueio global

O que é uma SAGA?

- Saga como transação distribuída composta
- Decomposição de uma transação longa em passos menores
- Cada passo possui uma ação e uma compensação
- Garantia de consistência eventual, não atomicidade forte
- O sistema aceita estados intermediários inválidos temporariamente
- Caminho bem definido de correção.

Origem

- Artigo de Garcia-Molina e Salem (1987)
- Transação longa “quebrada” em subtransações independentes
- Long-Lived Transactions (LLT)
- Evitar bloqueios prolongados de recursos
- Transações longas como composição de transações curtas

■

SAGAS

*Hector Garcia-Molina
Kenneth Salem*

Department of Computer Science
Princeton University
Princeton, N J 08544

Abstract

Long lived transactions (LLTs) hold on to database resources for relatively long periods of time, significantly delaying the termination of shorter and more common transactions. To

the majority of other transactions either because it accesses many database objects, it has lengthy computations, it pauses for inputs from the users, or a combination of these factors. Examples of LLTs are transactions to produce monthly account statements at a bank, transactions to



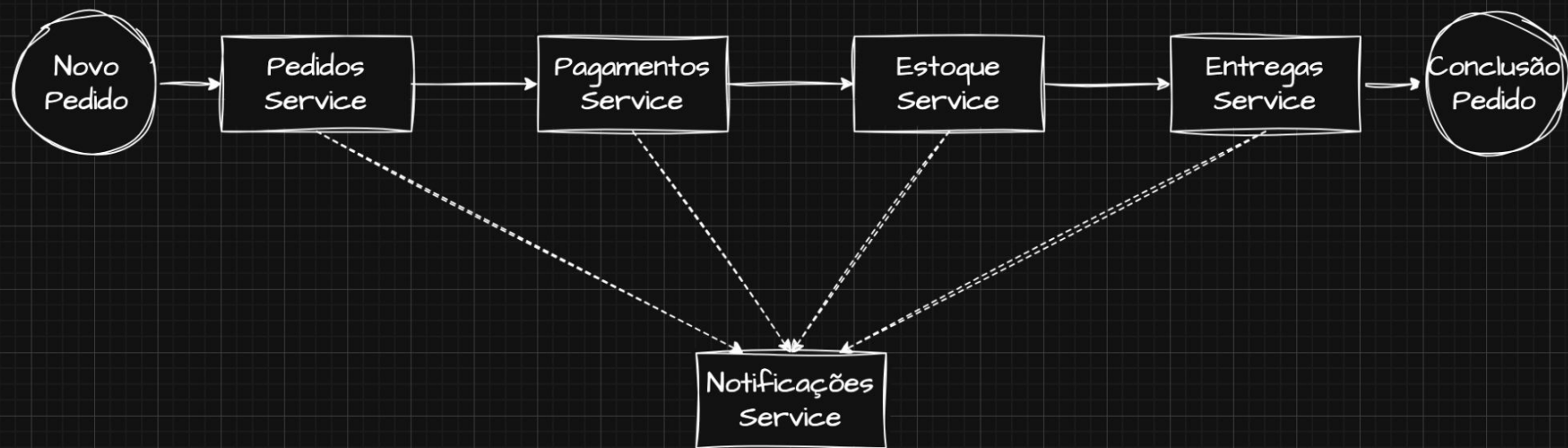
2 Problemas das Transações Distribuídas e Longas

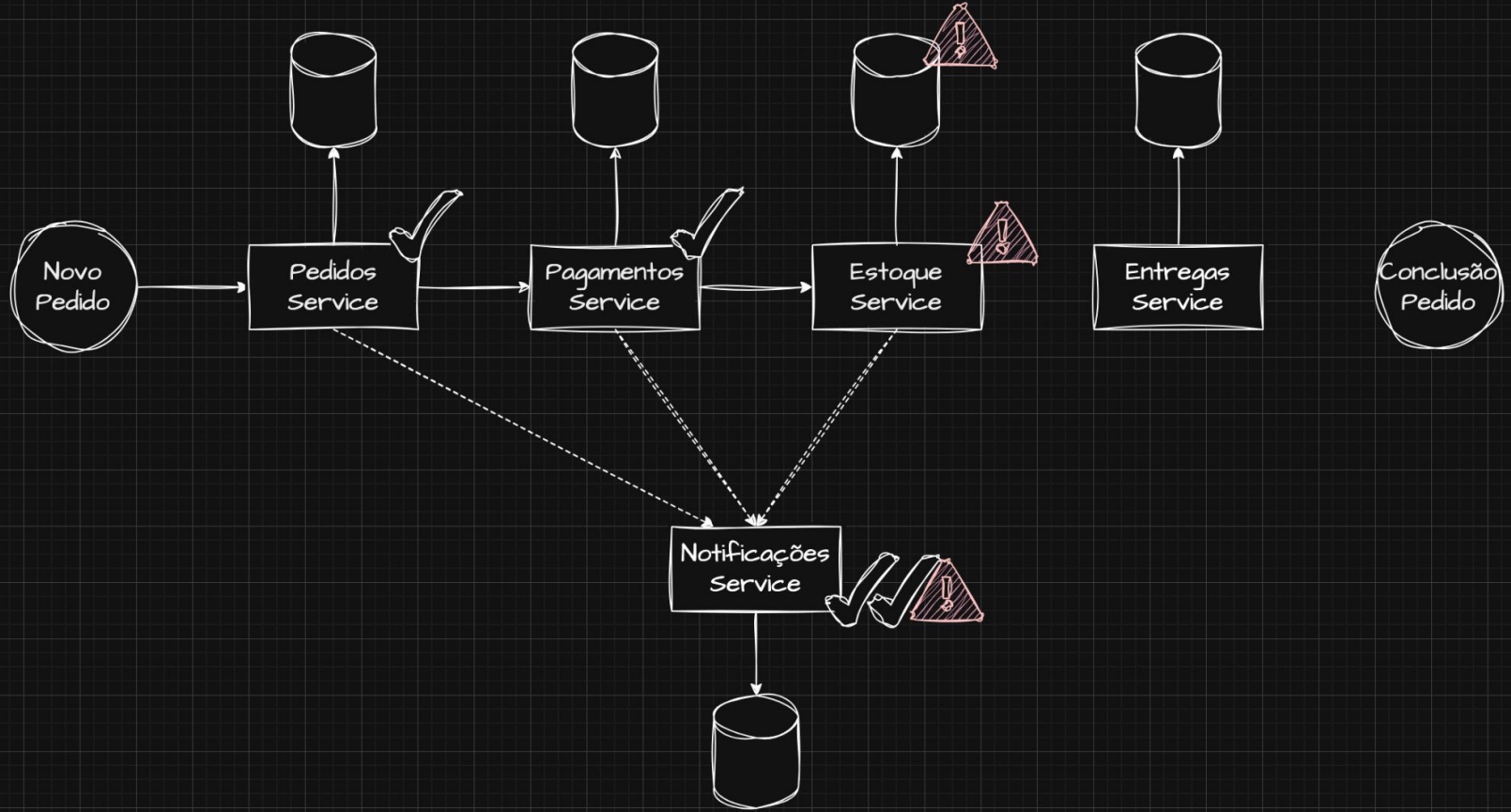


Definições e
Conceitos

0 Problema das Transações Distribuídas

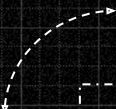
- Múltiplos participantes precisam confirmar sucesso
- Falha em qualquer ponto compromete a consistência
- Efeito cascata de erros
- Estados parcialmente confirmados
- Falhas em Silêncio
- Resíduos de estado inconsistente espalhados pelos serviços





0 Problema das Transações Longas

- Processos que duram segundos, horas ou dias
- Dependência de eventos externos
- Impossibilidade de manter locks
- Necessidade de persistir estado intermediário
- Linha do tempo longa com estados intermediários
- Billing, parcelamento, faturamento, uso de recursos, Onboarding e etc



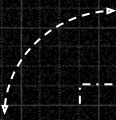
3 Proposta Arquitetural do Saga Pattern



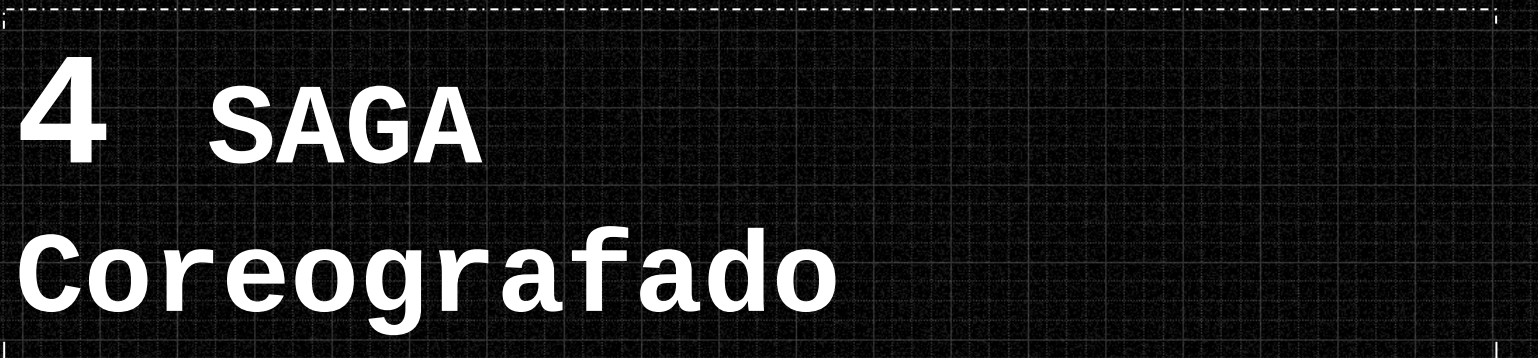
Definições e
Conceitos

Proposta Arquitetural do Saga Pattern

- "Conta uma História"
- Sequência coordenada de transações locais
- One Database Per Service
- Compensações explícitas
- Alternativa ao Two-Phase Commit
- Troca isolamento forte por escalabilidade
- Coreografado e Orquestrado



4 SAGA Coreografado

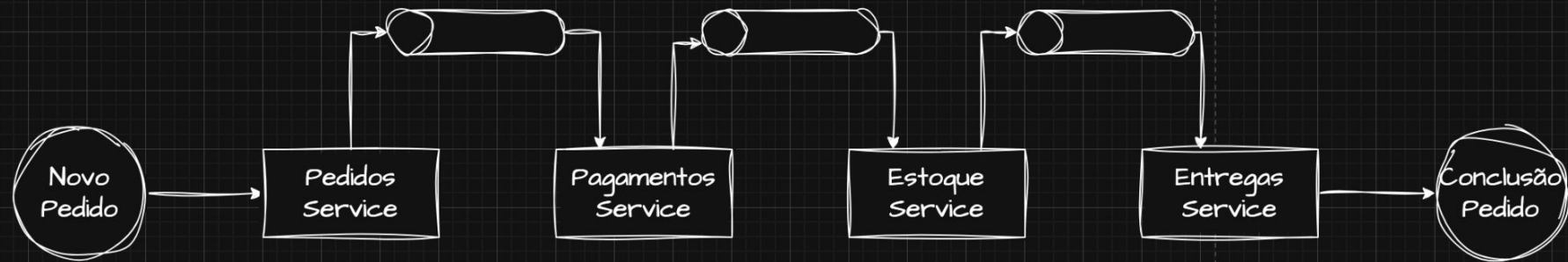


Definições e
Conceitos



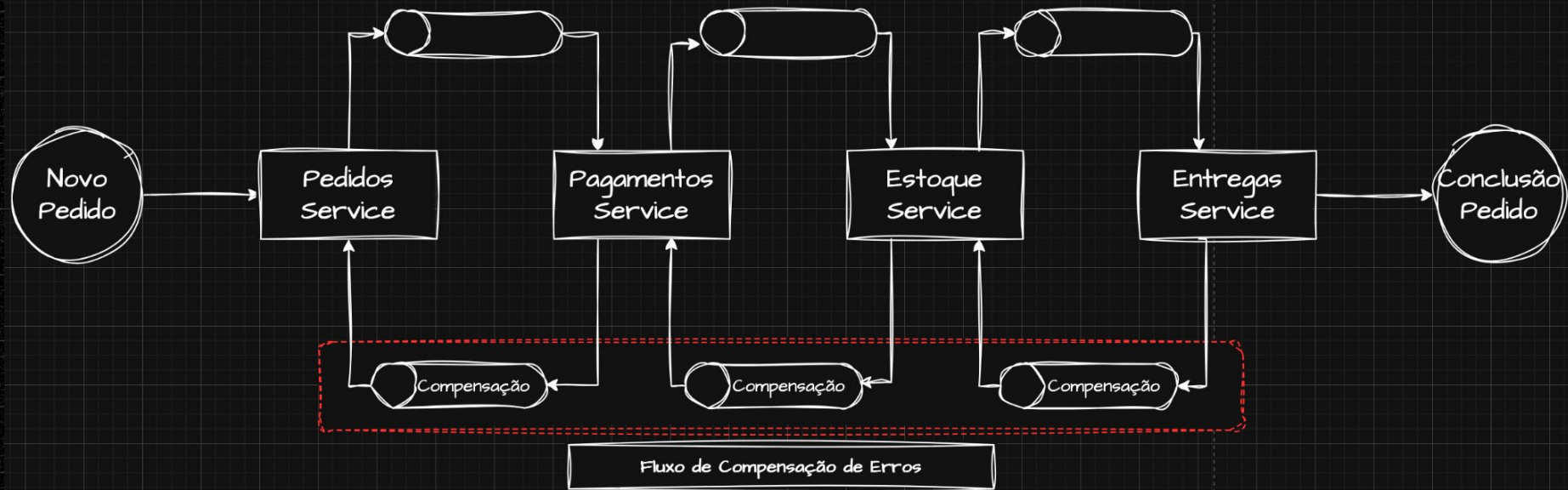
SAGA Coreografado

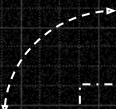
- Pipeline de Dados
- Não existe coordenador central
- Serviços conhecem próximos passos
- Comunicação por eventos
- Menor overhead inicial, maior acoplamento lógico
- Cadeia de serviços publicando eventos em sequência



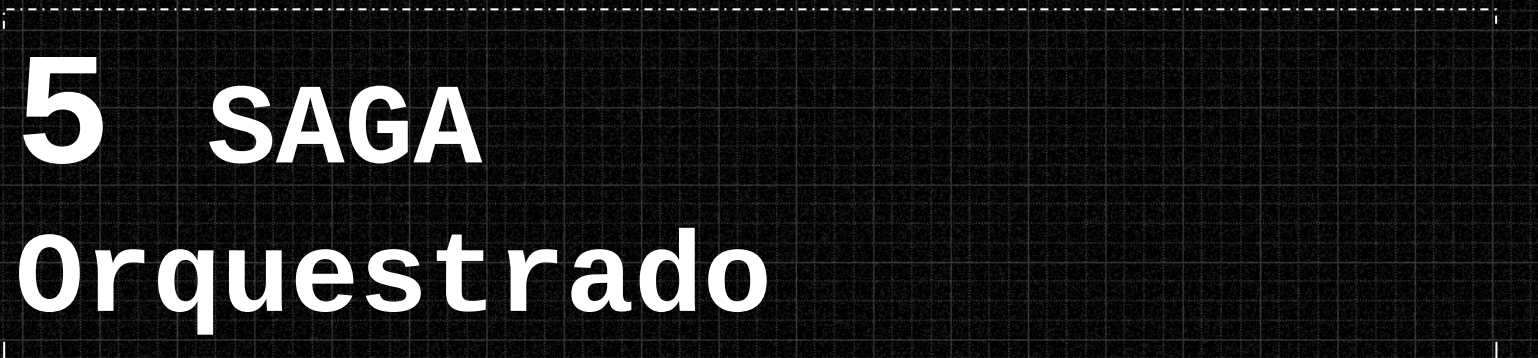
Coreografia e Compensação

- Serviço que falha aciona rollback
- Propagação reversa de compensações
- Dependência forte entre participantes
- Dificuldade de observabilidade global





5 SAGA Orquestrado

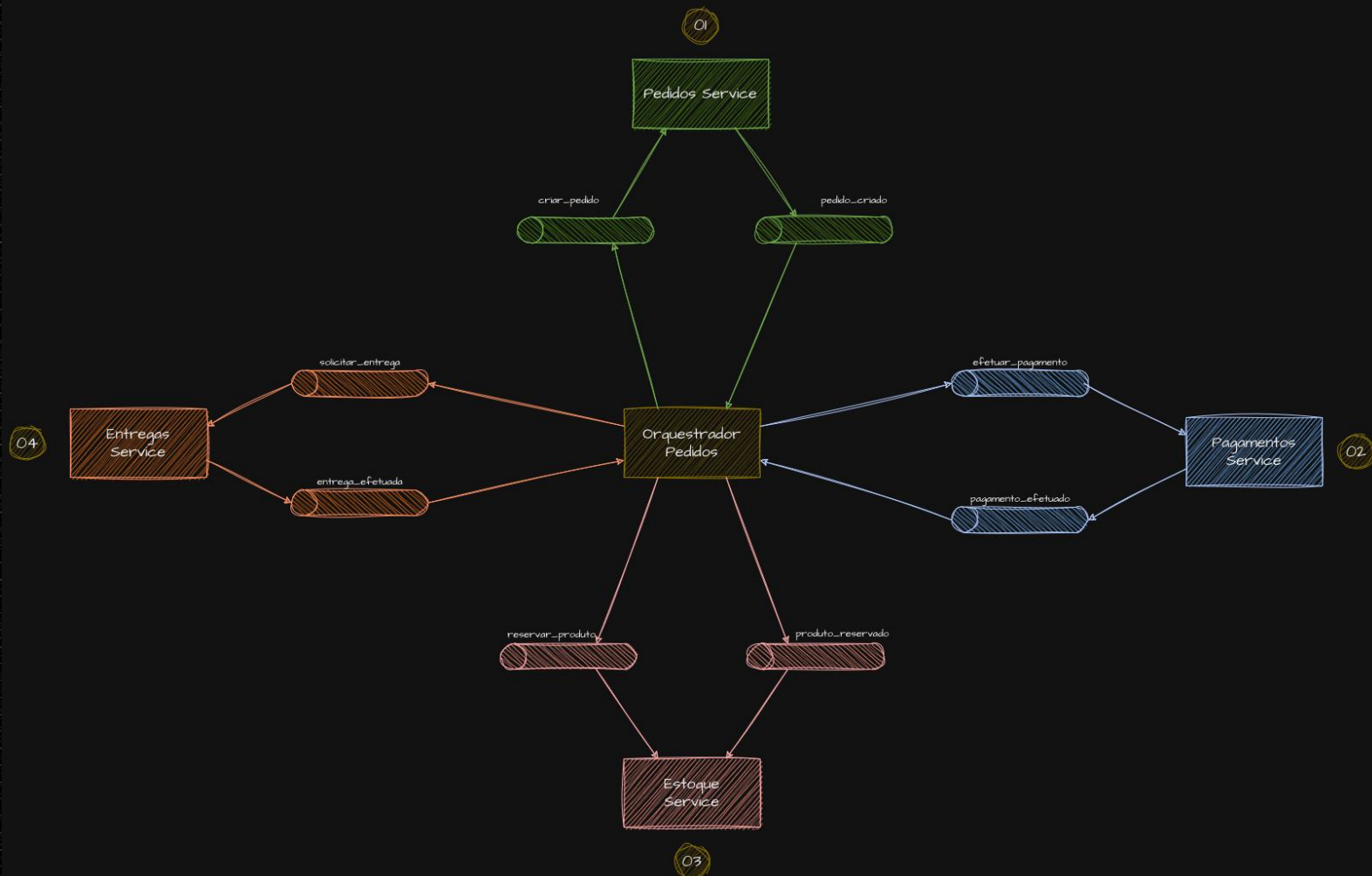


Definições e
Conceitos



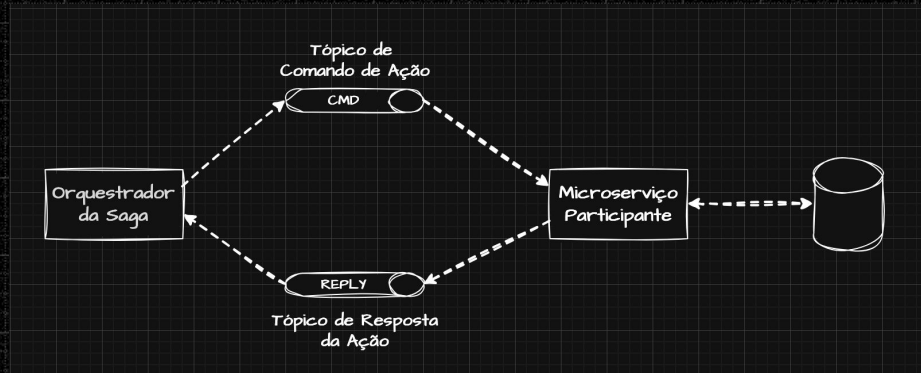
SAGA Orquestrado

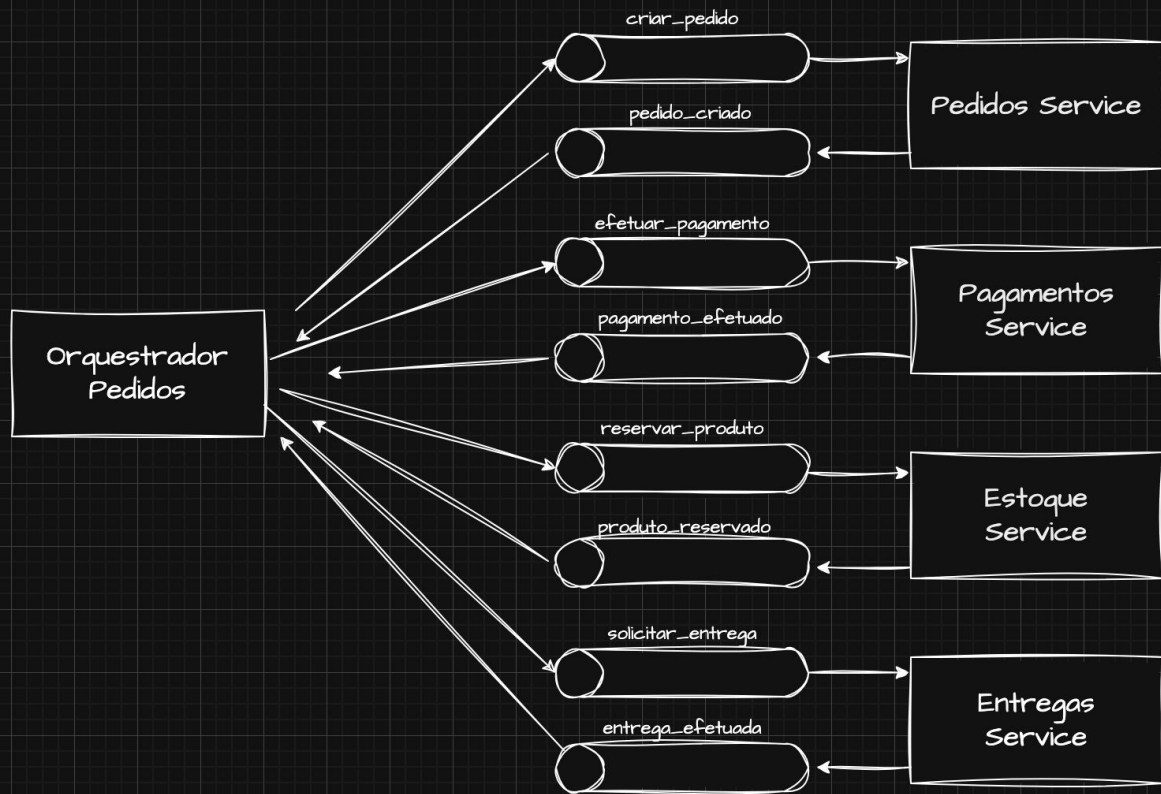
- Orquestrador central
- Workflow Pattern
- State Machine Pattern
- Command / Response
- Observabilidade e rastreabilidade centralizada
- Compensação Centralizada

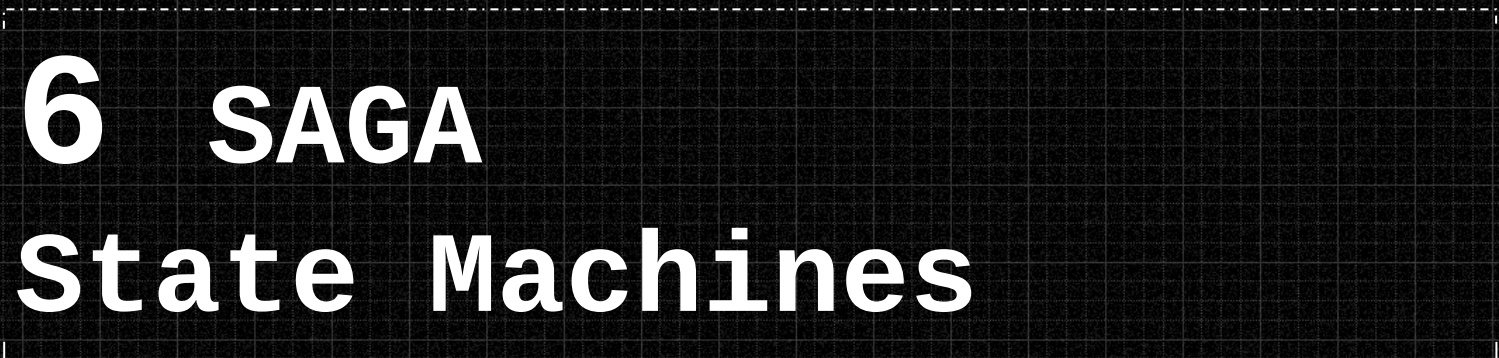
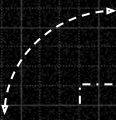


Command e Reply (Response)

- Tópicos de Ação e Reação
- Comandos acionam ações
- Resposta determinam as transições
- Comunicação Assíncrona







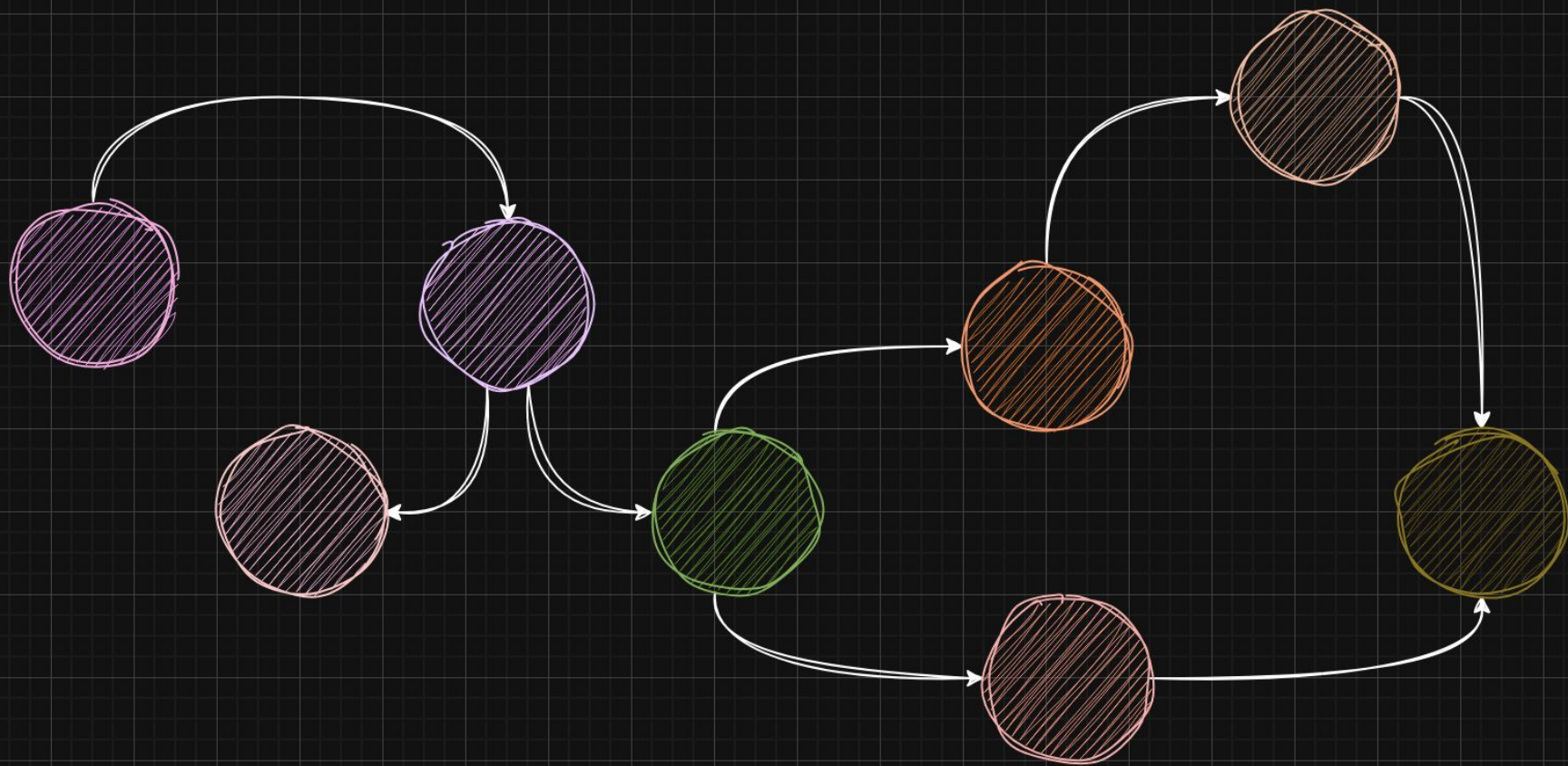
6 SAGA State Machines

Definições e
Conceitos



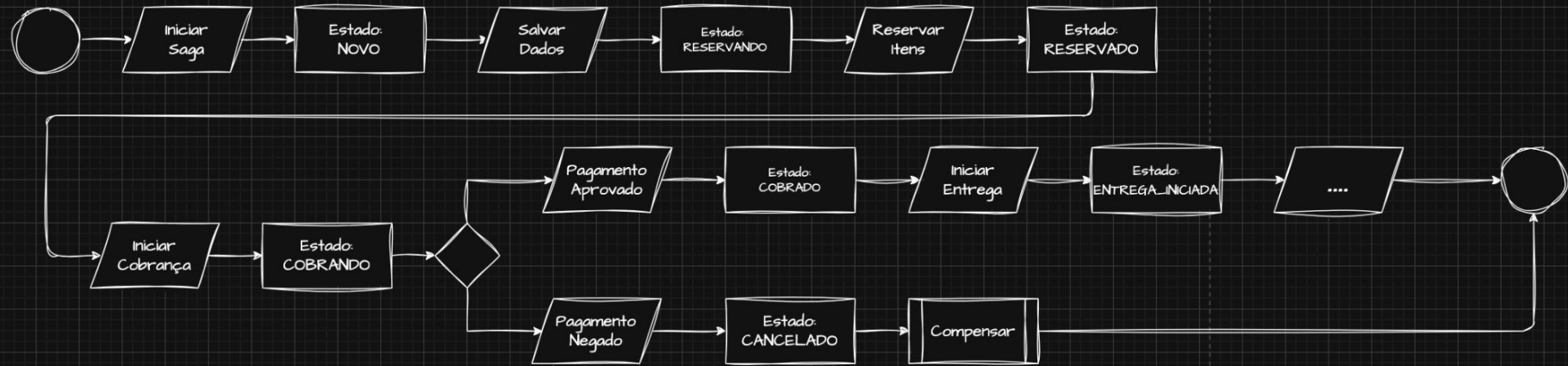
Maquinas de Estado

- Estado representa o progresso da saga
- Eventos disparam transições
- Transições executam ações
- Estado como fonte da verdade
- Estados Deterministicos e Especializados
- Event Sourcing Friendly



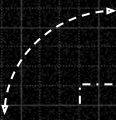
Ciclo de Vida da Saga

- Estados iniciais, intermediários e finais
- Caminho feliz e caminhos de erro
- Estados terminais explícitos
- Falha como estado válido



Ciclo de Vida da Saga

- Qual evento acabei de receber? → COBRADO COM SUCESSO
- Qual é o meu estado atual? → COBRANDO
- Se meu estado é COBRANDO e eu recebo COBRADO COM SUCESSO, para qual estado devo ir? → INICIAR_ENTREGA
- Qual ação devo tomar ao entrar no estado INICIAR_ENTREGA?
→ Notificar o sistema de entregas.



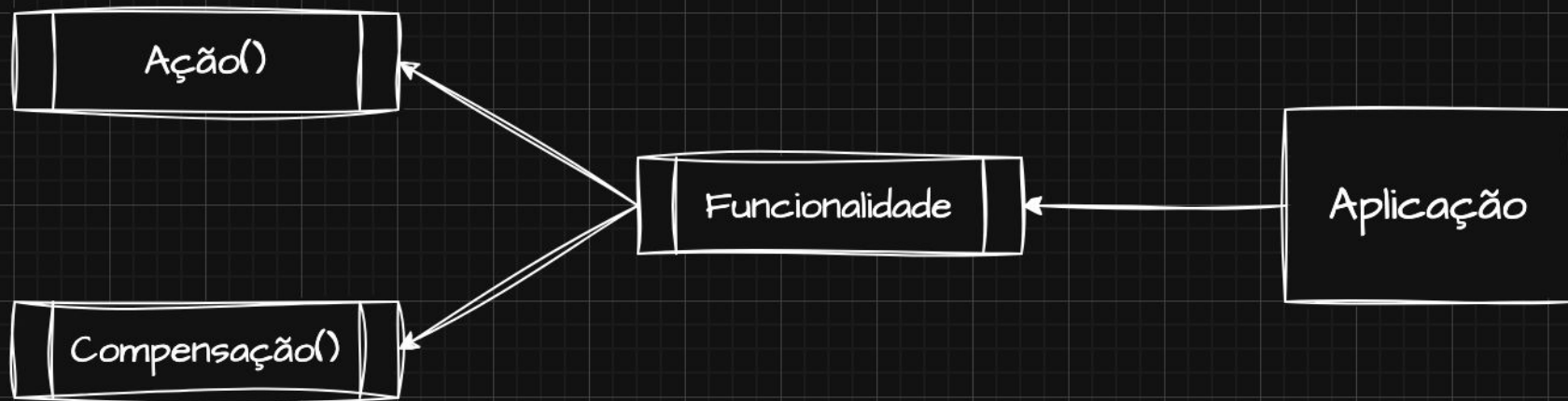
6 Modelo de Ação e Compensação

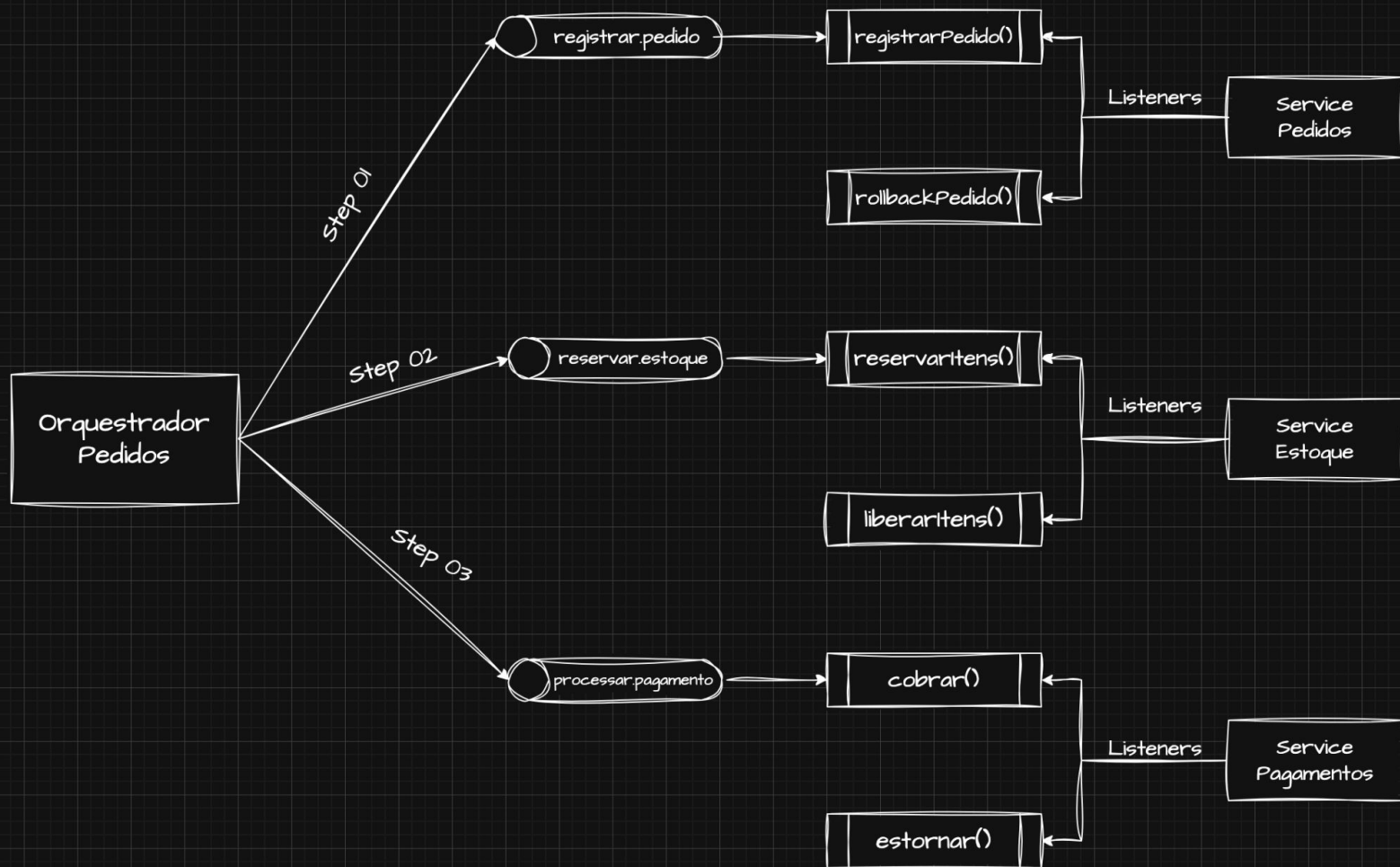
Definições e
Conceitos

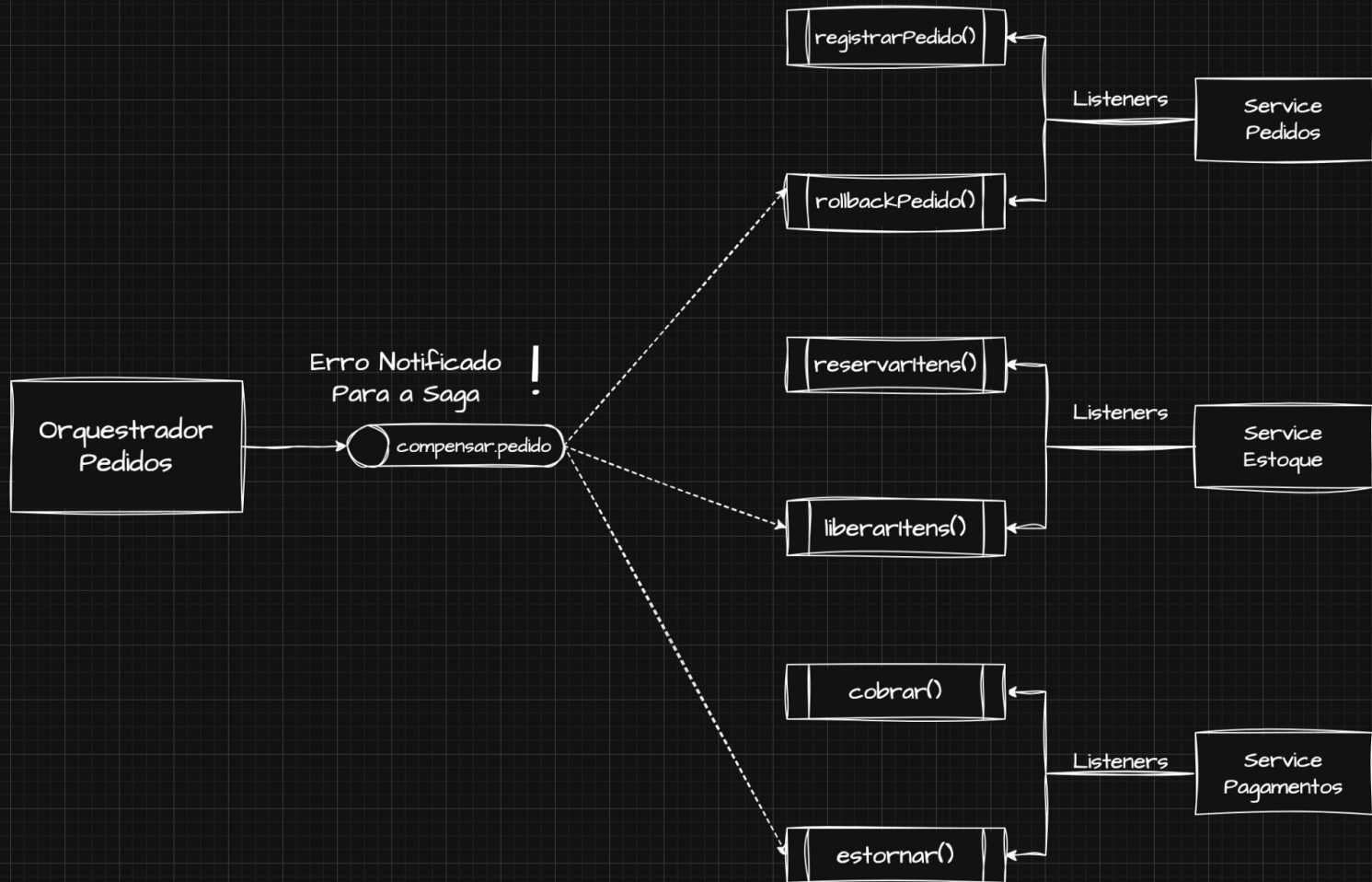


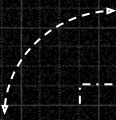
Modelo de Ação e Compensação

- Cada ação possui uma compensação
- Compensação $\neq$ rollback de banco
- Operações semanticamente inversas
- Design explícito de reversão
- Tópicos de Notificação
- Workflows de Desfazimento das Operações
- Consistência Distribuída









7 Logs de Transações

SAGA

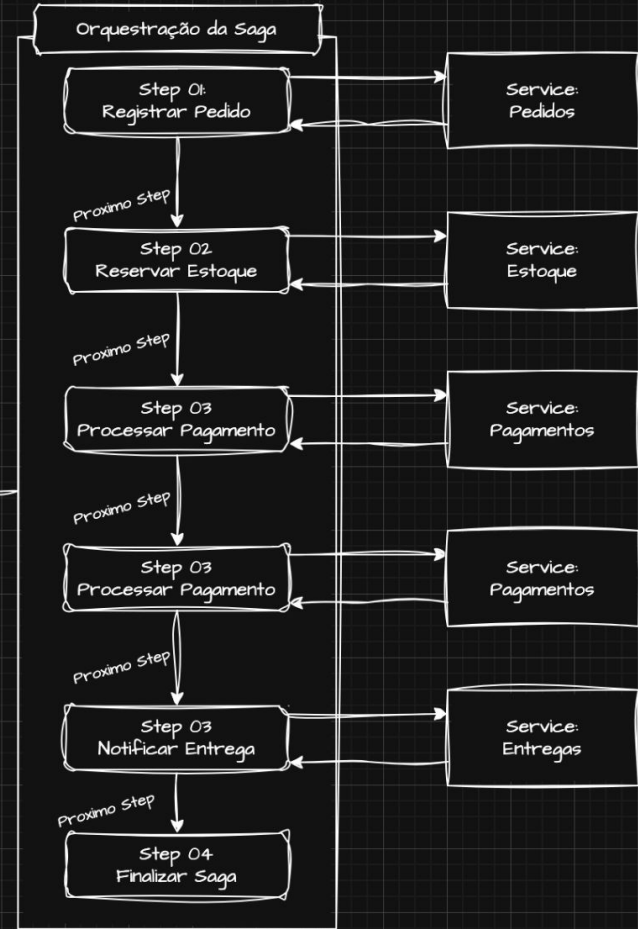


Definições e
Conceitos

Logs de Transações SAGA

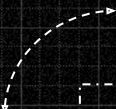
- Registro de cada passo executado
- Auditoria e debugging
- Reprocessamento e replay
- Reconstituição de Estado
- Base para observabilidade de negócio
- Event Sourcing Friendly

id_saga	Eventos	datetime	log	Status
123	Inicio da Saga	2024-12-09 12:55:45	{}	Concluido
123	Inicio Step 01	2024-12-09 13:00:02	{}	Concluido
123	Finalizar Step 01	2024-12-09 13:12:45	{}	Concluido
123	Inicio Step 02	2024-12-09 13:13:20	{}	Concluido
123	Finalizar Step 02	2024-12-09 13:14:33	{}	Concluido
123	Inicio Step 03	2024-12-09 13:14:45	{}	Concluido
123	Finalizar Step 03	2024-12-09 13:15:00	{}	Concluido
123	Finalizar Saga	2024-12-09 13:15:12	{}	Concluido



id_saga	Eventos	datetime	log	Status
123	Inicio da Saga	2024-12-09 12:55:45	{...}	Concluido
123	Inicio Step 01	2024-12-09 13:00:02	{...}	Concluido
123	Finalizar Step 01	2024-12-09 13:12:45	{...}	Concluido
123	Inicio Step 02	2024-12-09 13:13:20	{...}	Erro
123	Finalizar Step 02		{...}	Pendente
123	Inicio Step 03		{...}	Pendente
123	Finalizar Step 03		{...}	Pendente
123	Compensar Step 01	2024-12-09 13:15:02	{...}	Concluido
123	Step 01 Compensado	2024-12-09 13:20:15	{...}	Concluido
123	Finalizar Saga	2024-12-09 13:20:30	{...}	Concluido

Mudança de
Fluxo da Saga
para
Compensação



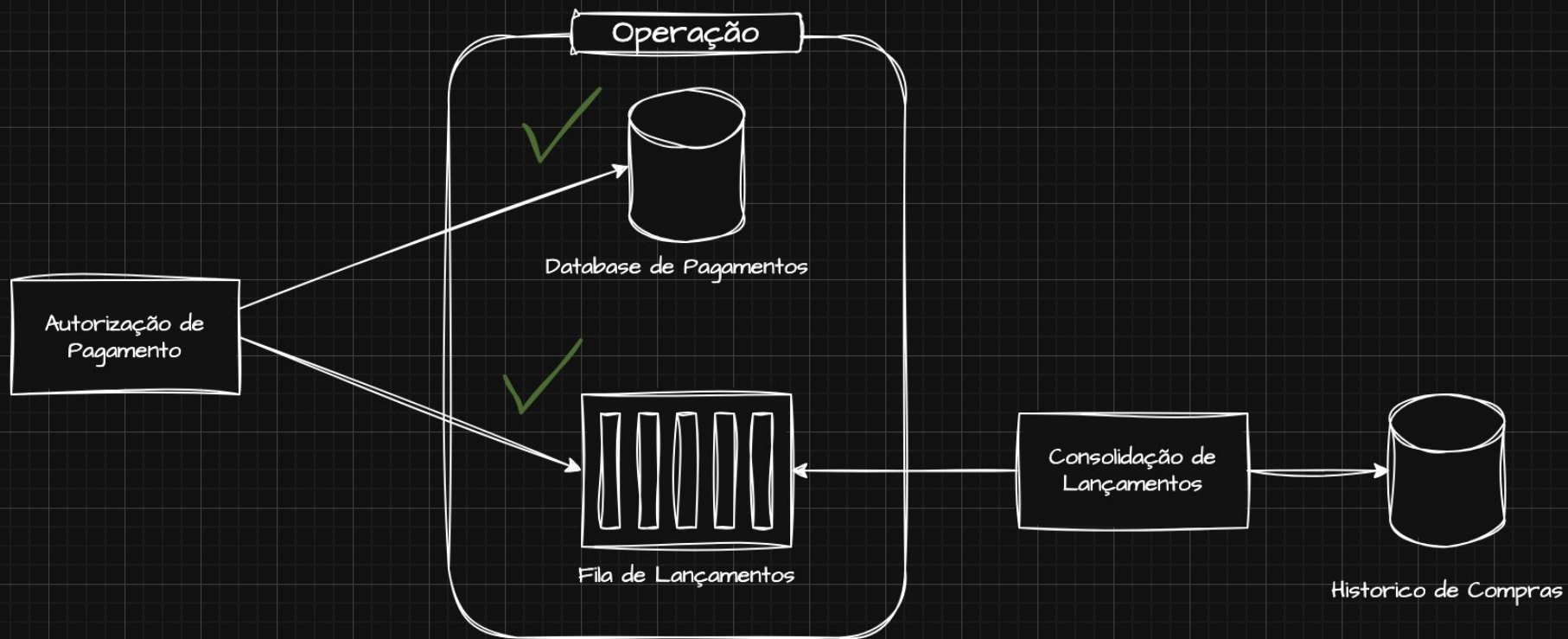
8 Dual-Write e Consistencia

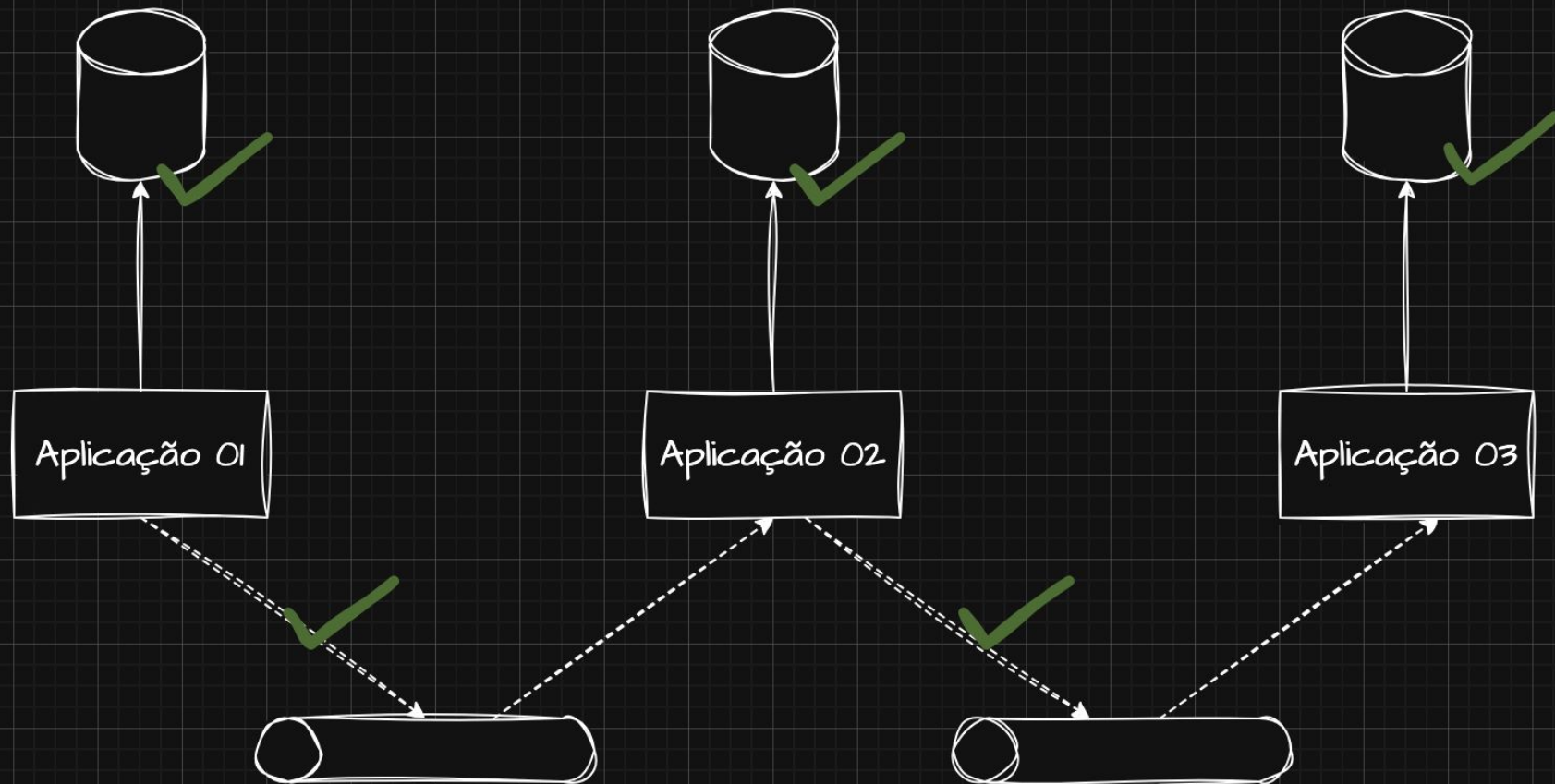


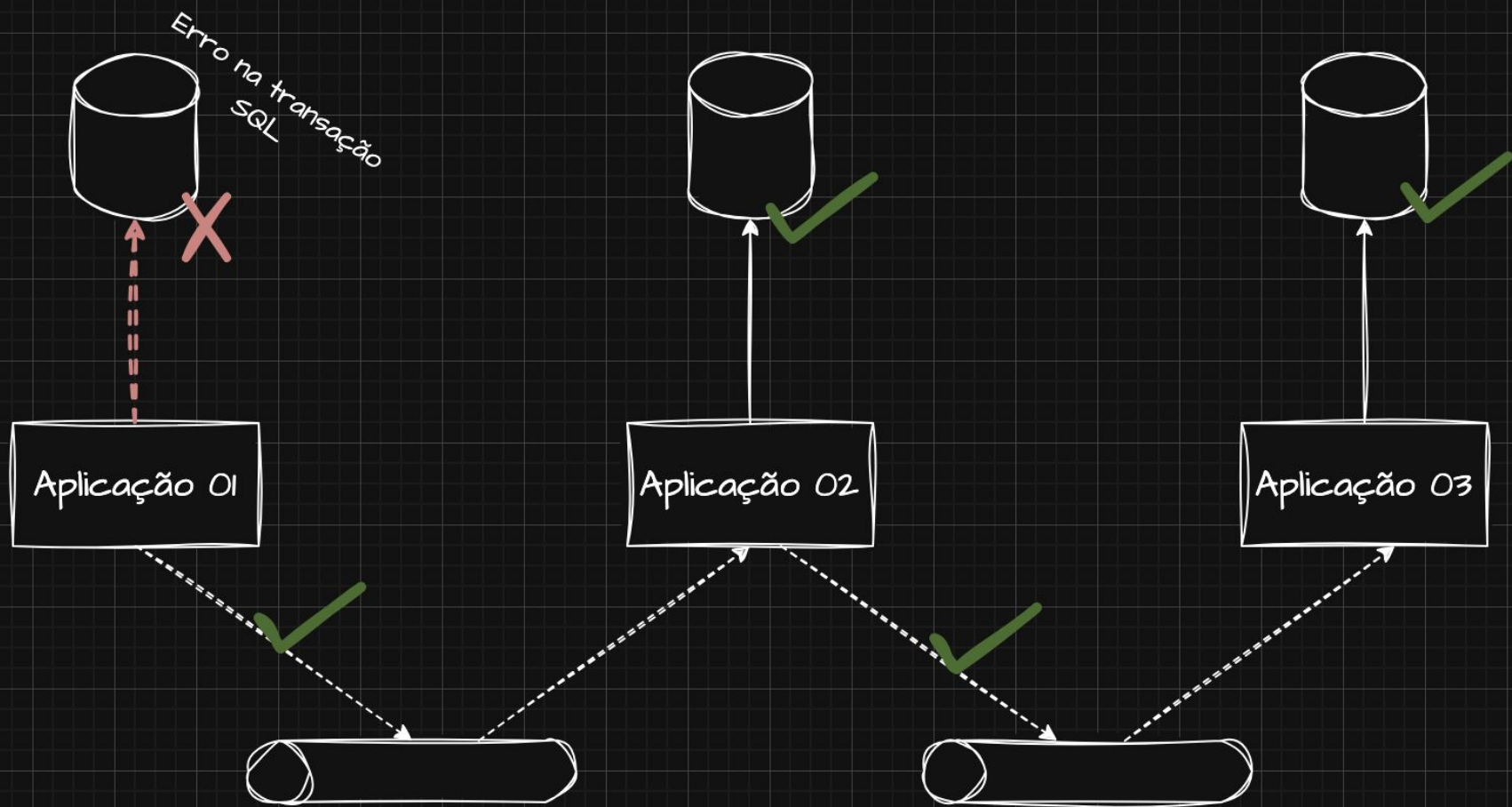
Definições e
Conceitos

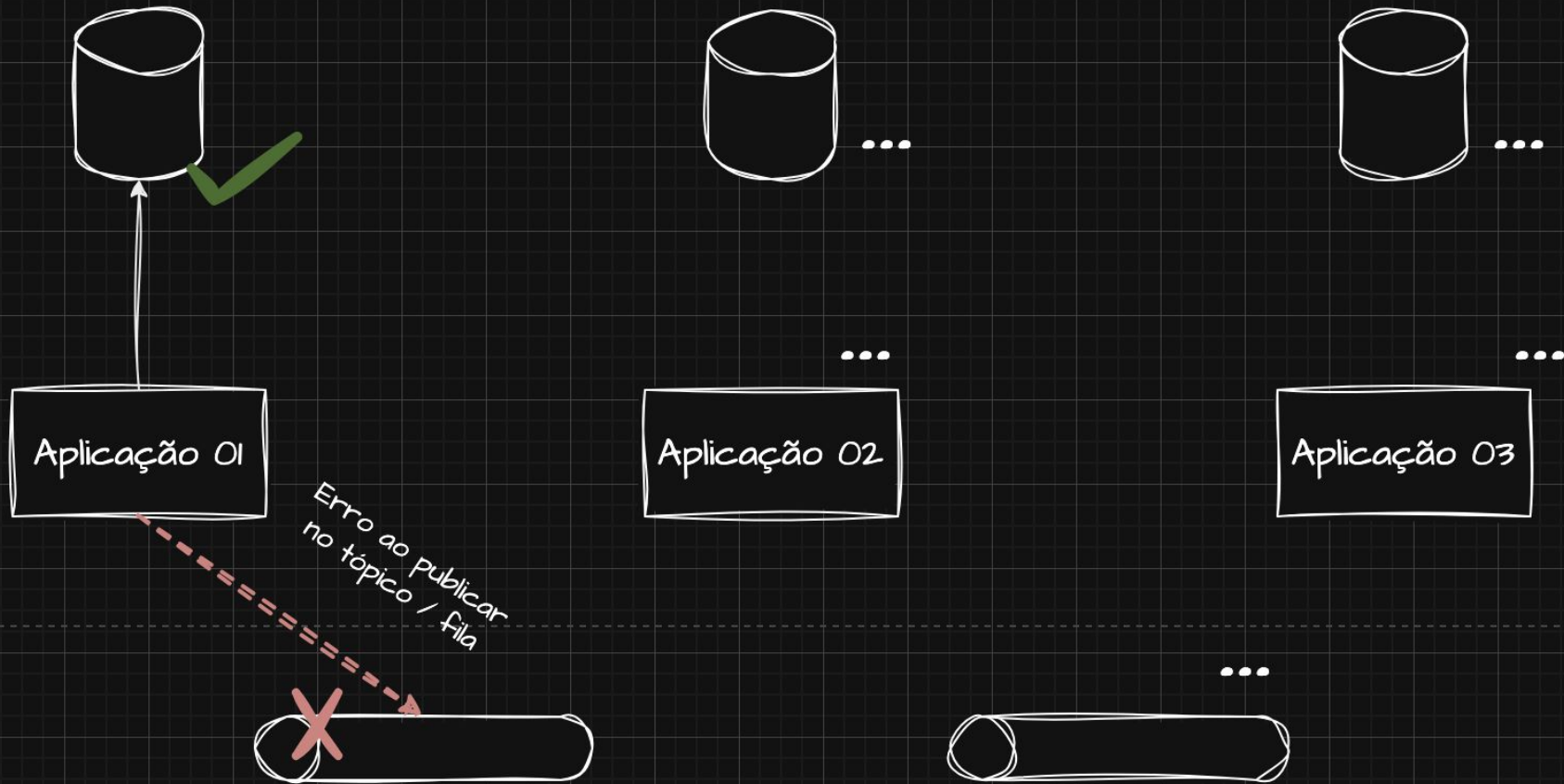
Dual Write

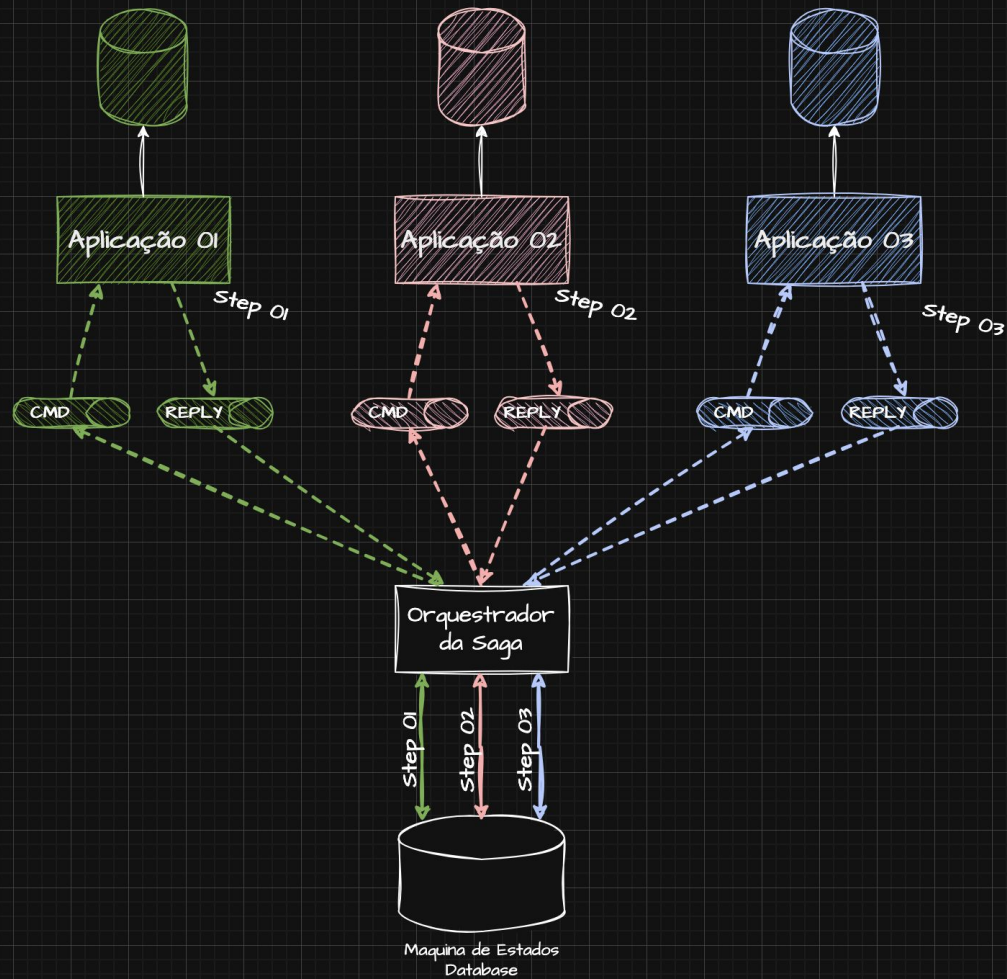
- Escrita em banco + evento
- Falha parcial gera inconsistência
- Presente em coreografia e orquestração
- Necessidade de atomicidade lógica

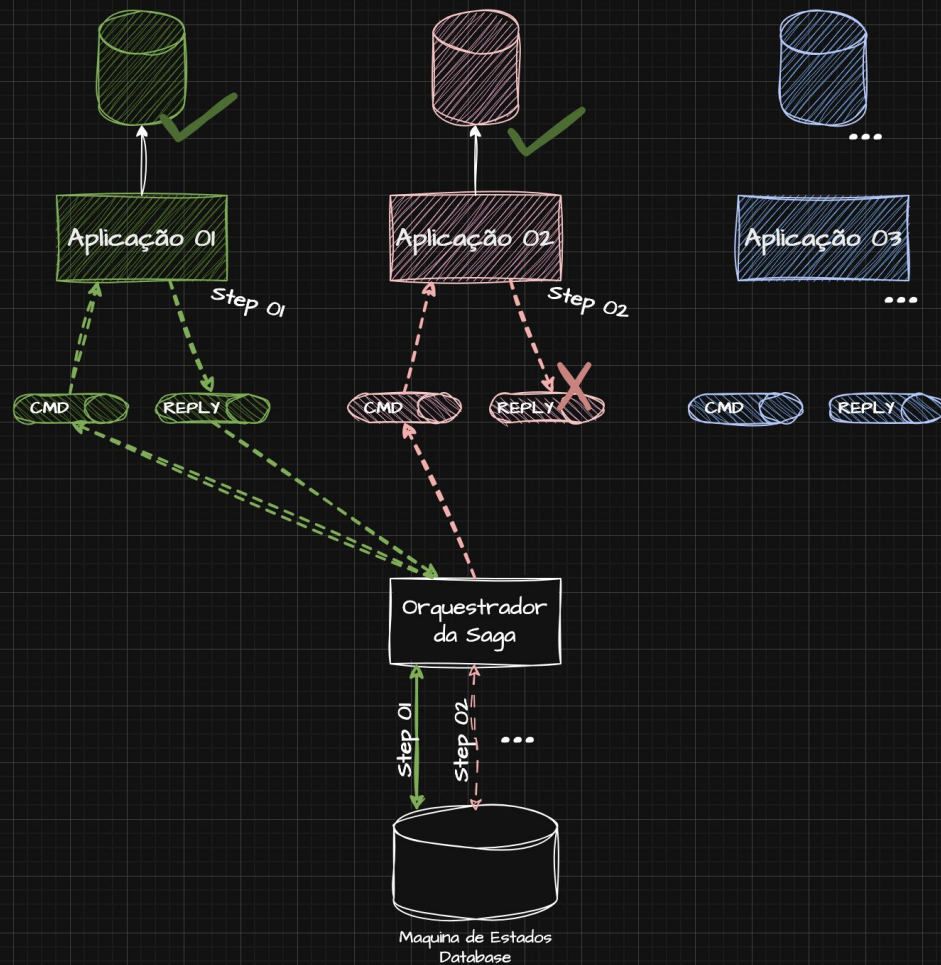












Outbox Pattern

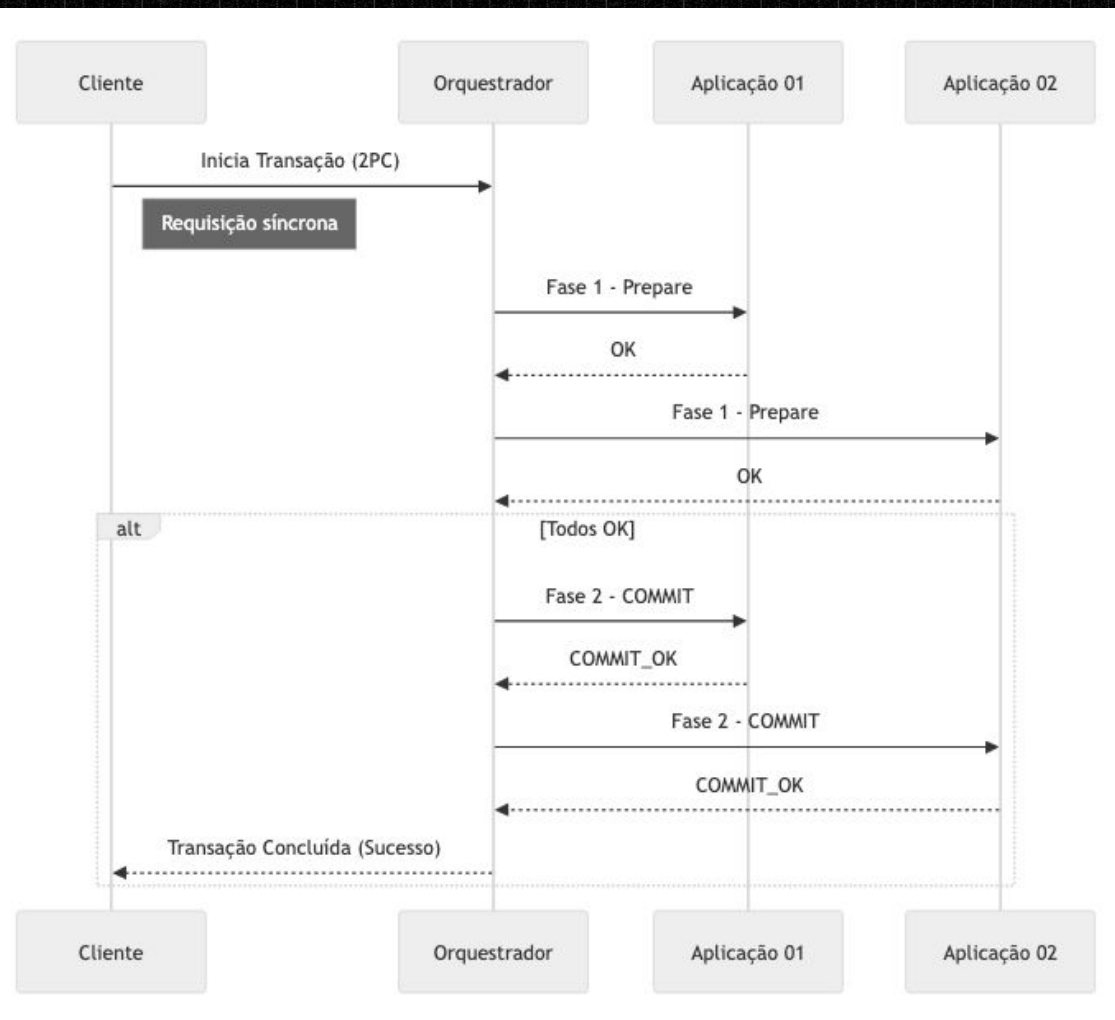
- Escrita do evento no mesmo commit do dado
- Relay assíncrono Garantia de entrega
- Redução drástica de inconsistências

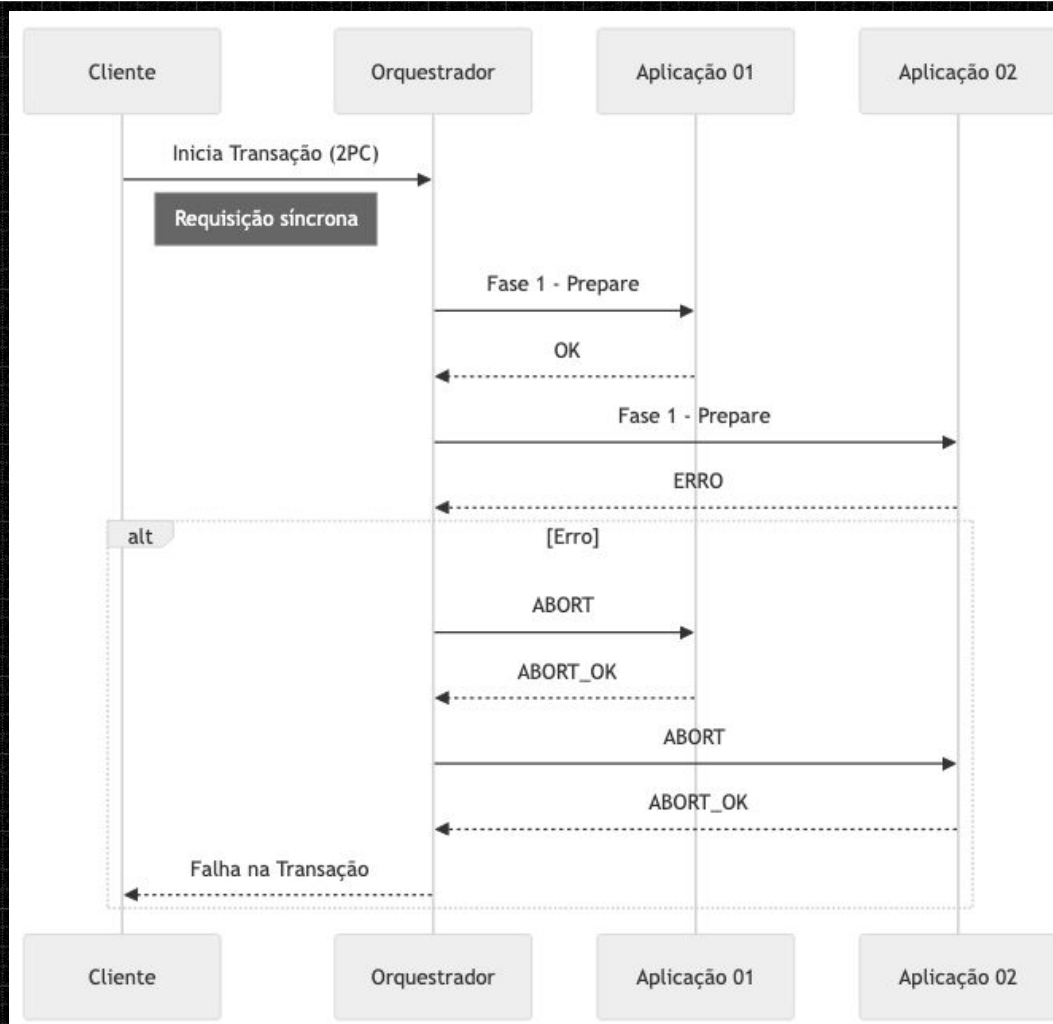
Outbox Pattern



Two-Step Commit

- Escrita do evento no mesmo commit do dado
- Relay assíncrono Garantia de entrega
- Redução drástica de inconsistências





\$ whoami

Matheus Fidelis

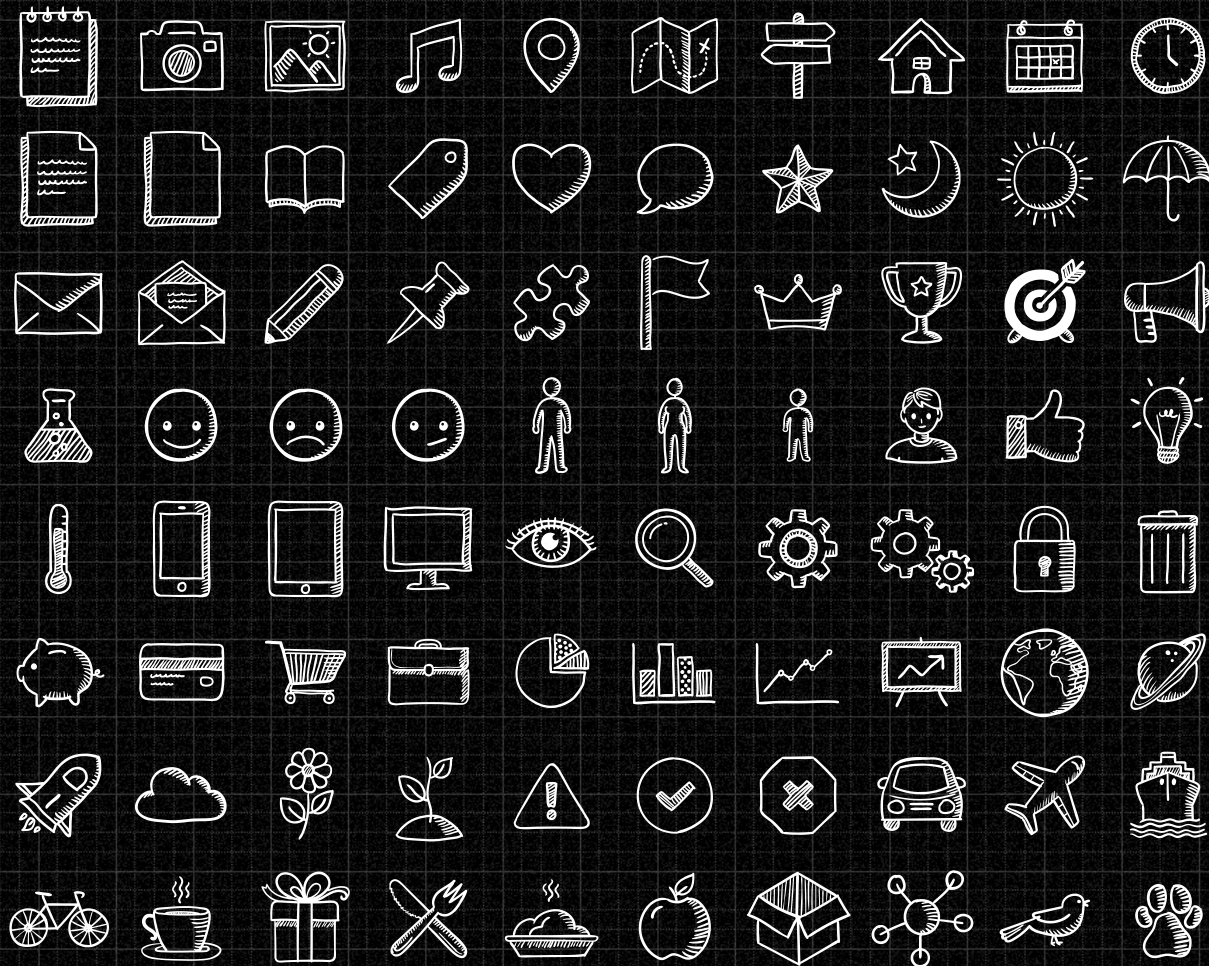
Engenheiro de \$RANDOM

@fidelissauro

<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>





SlidesCarnival icons are editable shapes.

This means that you can:

- Resize them without losing quality.
- Change fill color and opacity.

Isn't that nice? :)

Examples:

